Analisi Funzionale — v1.0

Functional Accelerator

Piattaforma AI-assisted per l'analisi degli impatti funzionali e UI/UX

Strumento interno che supporta i functional analyst nell'analisi degli impatti di un cambiamento software, sfruttando Claude (Anthropic) per estrarre impatti funzionali, regole di business, prototipi UI e valutazioni del rischio a partire dalla documentazione AS-IS e TO-BE del progetto.

VERSIONE DOCUMENTO	STATO	CONTESTO
1.0 — Aprile 2026	MVP — in produzione	Uso interno / consulting

INDICE

Sommario

1. Obiettivo e contesto

1.1 Problema risolto 3

1.2 Utenti target 3

1.3 Caso d'uso principale 3

2. Architettura generale

2.1 Stack tecnologico 4

2.2 Principi architetturali 4

2.3 Struttura dei moduli 4

3. Modello dati

3.1 Schema SQLite 5

3.2 Relazioni tra entità 5

4. Funzionalità — Gestione Progetti

5. Funzionalità — Upload documenti

6. Funzionalità — Analisi degli impatti (Claude)

7. Funzionalità — Prototipo UI per impatto

8. Funzionalità — Risk Assessment

9. API REST — Riferimento

10. Sicurezza e limitazioni MVP

11. Roadmap evolutiva

SEZIONE 1

Obiettivo e contesto

1.1 Problema risolto

Nei progetti di delivery software, la fase di analisi funzionale degli impatti — ovvero la comprensione di *cosa cambia* a seguito di nuovi requisiti — è tradizionalmente lenta, manuale e soggetta a omissioni. Un functional analyst deve leggere centinaia di pagine di documentazione (AS-IS + TO-BE), identificare gli impatti funzionali e UI, estrarre le regole di business e strutturare il tutto in un documento consegnabile.

Il Functional Accelerator automatizza questa fase sfruttando Claude (Anthropic) come motore di reasoning: il modello legge i documenti caricati e produce in pochi minuti un'analisi strutturata, completa di impatti, regole di business, schermate coinvolte e domande aperte.

1.2 Utenti target

- **Functional Analyst** — utilizzo primario: carica documenti, avvia analisi, revisiona e scarica il risultato
- **Business Analyst / Consultant** — revisiona gli impatti, usa la funzione prototipo UI per comunicare le modifiche ai clienti
- **QA Engineer / Test Manager** — usa il Risk Assessment per pianificare i test in un nuovo deployment a partire dai defect storici

1.3 Caso d'uso principale

1. **Crea progetto** — assegna nome e descrizione al contesto di analisi (es. "KFC Austria → Slovakia migration")
2. **Carica documenti** — AS-IS (stato corrente), TO-BE (requisiti target) e opzionalmente un file di Business Rules fornite (factum positum)
3. **Avvia analisi** — Claude elabora i documenti in background e produce una struttura JSON normalizzata
4. **Revisiona i risultati** — executive summary, impatti funzionali, impatti UI/UX, schermate coinvolte, regole di business, domande aperte
5. **Genera prototipo UI** — per ogni impatto UI/UX, carica uno screenshot AS-IS e ottieni un'immagine modificata con il cambiamento visualizzato
6. **Risk Assessment** — carica l'export ALM dei defect del progetto sorgente per prevenire problemi nel deployment target

SEZIONE 2

Architettura generale

2.1 Stack tecnologico

FRONTEND

React 18 | Vite | TypeScript | Tailwind CSS v3 | React Router v7 | Axios | DOMPurify | Lucide Icons

SPA servita da Vite dev server (porta 5173). Comunicazione con il backend esclusivamente via API REST.

BACKEND

Node.js | Express 4 | TypeScript | ts-node-dev | better-sqlite3 | multer | Puppeteer

API server su porta 3001. Il DB è un file SQLite locale (`./data/app.db`). Tutte le query DB sono sincrone (better-sqlite3).

AI

@anthropic-ai/sdk | claude-opus-4-5

Il backend chiama direttamente l'API Anthropic. Il frontend non contatta mai Claude. La chiave API è in `.env`, mai nel codice.

PARSING FILE

pdf-parse | mammoth | xlsx

Estrazione testo da PDF, DOCX e Excel. Le immagini caricate non estraggono testo, ma vengono passate come vision block a Claude.

2.2 Principi architetturali

- **Clean separation:** il frontend non chiama mai Claude direttamente; tutta la logica AI vive nel backend.
- **AS-IS / TO-BE / Business Rules come first-class concept:** ogni file ha un bucket che determina come viene usato nel prompt.
- **Analisi asincrona con polling:** il backend risponde 202 immediatamente e aggiorna lo stato nel DB; il frontend interroga ogni 2 secondi.
- **Prompt strutturati:** tutti i prompt sono costruiti da `promptBuilder.ts`, mai inline.
- **SQLite sincrono:** better-sqlite3 usa API sync, nessun async database chaos.
- **WAL mode attivo:** migliore performance in lettura concorrente; foreign keys abilitate.

2.3 Struttura dei moduli backend

File	Responsabilità
`src/server.ts`	Entry point, middleware, routing principale
`src/db/index.ts`	Init SQLite, schema, migration automatica
`src/routes/projects.ts`	CRUD progetti

File	Responsabilità
`src/routes/files.ts`	Upload, lista, delete, preview file
`src/routes/analysis.ts`	Trigger analisi, polling, prototipo UI per impatto
`src/routes/risk.ts`	Risk Assessment: upload defect file, esecuzione asincrona, storico
`src/services/claude.ts`	Client Anthropic: `callClaude`, `callClaudeForHtml`, `callClaudeJson`
`src/services/promptBuilder.ts`	Costruzione prompt: analisi principale, prototipo UI, risk assessment
`src/services/fileParsing.ts`	Estrazione testo da PDF/DOCX/Excel/TXT; lettura immagini base64
`src/services/imageRenderer.ts`	Rendering HTML → PNG via Puppeteer headless

SEZIONE 3

Modello dati

3.1 Schema SQLite

Tabella: `projects`

Colonna	Tipo	Note
id	TEXT PK	UUID v4
name	TEXT NOT NULL	Nome del progetto
description	TEXT	Descrizione opzionale
status	TEXT	draft | ready | analyzing | done | error
created_at	TEXT	ISO datetime UTC
updated_at	TEXT	ISO datetime UTC

Tabella: `files`

Colonna	Tipo	Note
id	TEXT PK	UUID v4
project_id	TEXT FK → projects	ON DELETE CASCADE
original_name	TEXT	Nome file originale (display)
name	TEXT	Nome file su disco (uuid + ext)
mime_type	TEXT	MIME type rilevato da multer
size	INTEGER	Byte
bucket	TEXT	as-is | to-be | business-rules
path	TEXT	Path assoluto su filesystem
extracted_text	TEXT NULLABLE	Testo estratto (PDF/DOCX/Excel/TXT), capped a 100k chars
created_at	TEXT	ISO datetime UTC

Tabella: `analyses`

Colonna	Tipo	Note
id	TEXT PK	UUID v4
project_id	TEXT FK → projects	ON DELETE CASCADE

Colonna	Tipo	Note
version_name	TEXT	Auto-generato: "Analysis v1", "Analysis v2"...
status	TEXT	pending | running | done | error
input_summary	TEXT NULLABLE	Riepilogo input (nome progetto, n. file)
result_json	TEXT NULLABLE	JSON serializzato di AnalysisResult
error_message	TEXT NULLABLE	Messaggio di errore se status=error
created_at	TEXT	ISO datetime UTC

Tabella: `impact_prototypes`

Colonna	Tipo	Note
id	TEXT PK	UUID v4
analysis_id	TEXT FK → analyses	ON DELETE CASCADE
impact_id	TEXT	Es. "UX-01" — unico per analysis_id
image_data	TEXT	Immagine PNG in base64 (generata da Puppeteer)
created_at	TEXT	ISO datetime UTC

Tabella: `risk_assessments`

Colonna	Tipo	Note
id	TEXT PK	UUID v4
project_id	TEXT FK → projects	ON DELETE CASCADE
version_name	TEXT	Auto-generato: "Risk v1", "Risk v2"...
status	TEXT	pending | running | done | error
defect_count	INTEGER NULLABLE	N. di defect nel file caricato
result_json	TEXT NULLABLE	JSON serializzato di RiskAssessmentResult
error_message	TEXT NULLABLE	
progress_step	TEXT NULLABLE	Label step corrente (es. "Categorizing 87 defects...")
created_at	TEXT	ISO datetime UTC

3.2 Schema JSON — AnalysisResult

> Il campo `result_json` della tabella `analyses` contiene un oggetto JSON con questo schema fisso: `executiveSummary`, `functionalImpacts[]`, `uiUxImpacts[]`, `affectedScreens[]`, `businessRulesExtracted[]`, `proposedChanges[]`, `assumptions[]`, `openQuestions[]`. Ogni

Impact ha: `id` (es. "FI-01"), `area`, `description`, `severity` (high|medium|low). I BusinessRule hanno: `id`, `description`, `source` (provided|as-is|to-be|inferred).

SEZIONE 4

Funzionalità — Gestione Progetti

Un **Progetto** è il contenitore principale di tutto il lavoro di analisi. Raggruppa documenti, analisi e risk assessment relativi a un singolo contesto (es. "KFC Austria → Slovakia").

4.1 Creazione progetto

Dall'homepage, il pulsante **New Project** apre un modal che richiede nome (obbligatorio) e descrizione (opzionale). Il progetto viene creato con status `draft`.

4.2 Modifica progetto

Dalla pagina di dettaglio, il pulsante **Edit** nell'header apre una modalità inline che permette di modificare nome e descrizione senza uscire dalla pagina. Le modifiche vengono salvate via `PATCH /api/projects/:id`.

4.3 Lista e navigazione

L'homepage mostra tutti i progetti in ordine cronologico inverso. Ogni card mostra nome, descrizione, status badge, numero di file e analisi associate. Click sulla card naviga al dettaglio.

4.4 Eliminazione progetto

Eliminare un progetto rimuove in cascade tutti i file (anche dal filesystem), le analisi, i prototipi e i risk assessment associati.

STATUS PROGETTO

- `draft` — appena creato, nessuna analisi
- `analyzing` — analisi in corso
- `done` — almeno un'analisi completata
- `error` — ultima analisi fallita

REGOLE DI BUSINESS

- Il nome progetto è obbligatorio (min 1 carattere)
- La descrizione è opzionale ma raccomandata
- Nessun limite al numero di progetti
- I file vengono salvati in `backend/uploads/{projectId}/`

SEZIONE 5

Funzionalità — Upload documenti

Ogni progetto supporta tre tipologie di documenti, distinguibili per bucket. Il bucket determina come il file viene usato nella costruzione del prompt per Claude.

5.1 Bucket dei file

Bucket	Badge	Scopo	Uso nel prompt
as-is	As-Is	Documentazione dello stato attuale del sistema	Sezione "AS-IS Documentation (Current State)"
to-be	To-Be	Requisiti e specifiche del target	Sezione "TO-BE Documentation (Target Requirements)"
business-rules	Business Rules	Regole di business fornite come factum positum	Sezione separata; Claude le tratta come autoritative senza re-inferire. Nel risultato, source = "provided".

5.2 Formati supportati

- **Documenti:** PDF, DOCX, DOC, TXT, MD — testo estratto e incluso nel prompt (cap 10.000 chars/file)
- **Fogli di calcolo:** XLSX, XLS — ogni sheet viene convertito in CSV e incluso nel prompt (cap 50.000 chars)
- **Immagini:** PNG, JPG, JPEG, GIF, WEBP — nessuna estrazione testo; per i file nel bucket business-rules vengono ignorati; usati come vision block solo nella funzionalità di prototipo UI

5.3 Comportamento upload

- Upload via drag&drop o file picker (multiplo)
- Progress bar per ogni file durante il caricamento
- Limite dimensione: 20 MB per file (configurabile via `MAX_FILE_SIZE_MB`)
- Il file viene salvato su disco e il testo estratto in modo asincrono prima di rispondere al client
- I file sono visualizzabili nella sezione Documents con anteprima per le immagini

> **Nota:** I file caricati sono archiviati in `backend/uploads/`, che è gitignored. Non vengono mai committati. In caso di reset del filesystem, i file devono essere ricaricati.

SEZIONE 6

Funzionalità — Analisi degli impatti

6.1 Flusso di esecuzione

1. L'utente clicca **Analyze Impacts**. Il backend crea un record `analyses` con status `running` e risponde 202 immediatamente.
2. In background, `runAnalysisAsync` costruisce il prompt via `buildAnalysisPrompt()` includendo tutti i file del progetto nelle sezioni appropriate.
3. Il prompt viene inviato a Claude (`claude-opus-4-5`) con `max_tokens: 8192` e timeout di 4 minuti.
4. Il frontend fa polling ogni 2 secondi su `GET /api/projects/:id` finché lo status non diventa `done` o `error`.
5. Al completamento, il JSON risultante viene salvato in `result_json` e il frontend renderizza i tab di analisi.

6.2 Output strutturato

Claude restituisce un JSON con i seguenti componenti, visualizzati in tab separati:

Tab	Campo JSON	Descrizione
Executive Summary	executiveSummary, assumptions, businessRulesExtracted	Sintesi narrativa (3-5 frasi), assunzioni fatte, regole di business estratte (con source: provided|inferred|as-is|to-be)
Functional Impacts	functionalImpacts[]	Impatti su logiche, processi, integrazioni. Ogni item: id (FI-xx), area, description, severity
UI/UX Impacts	uiUxImpacts[], proposedChanges[]	Impatti su interfacce utente. Include anche la tabella dei proposed changes per schermata
Affected Screens	affectedScreens[]	Schermate coinvolte con currentBehavior, proposedBehavior e changeType (modified|new|removed)
Open Questions	openQuestions[]	Domande aperte che il functional analyst deve risolvere prima di procedere

6.3 Gestione Business Rules

> Se il progetto contiene file nel bucket **business-rules**, il prompt include una sezione specifica che istruisce Claude a trattarle come *factum positum*: vengono incluse nel risultato con `source:`

`"provided"` senza essere riformulate. Le regole aggiuntive inferite dal contesto vengono marcate con `source: "inferred"`. L'executive summary distingue esplicitamente le due fonti.

6.4 Storico delle analisi

Ogni esecuzione crea una nuova versione ("Analysis v1", "v2"...). Tutte le versioni sono accessibili dal pannello di sinistra nella scheda *Analyses*. È possibile eliminare singole versioni. Non è possibile modificare i risultati di un'analisi completata: è necessario rieseguirla.

SEZIONE 7

Funzionalità — Prototipo UI per impatto

Per ogni impatto UI/UX identificato nell'analisi, è possibile generare una visualizzazione del cambiamento come immagine PNG.

7.1 Flusso

1. Nel tab **UI/UX Impacts**, l'utente espande un impatto cliccandoci sopra (accordion).
2. Compare un'area di upload dove caricare uno screenshot AS-IS della schermata attuale (PNG/JPG/WEBP, max 20MB).
3. Il click su **Generate Modified Screen** invia al backend: immagine + impactId + area + description.
4. Il backend chiama Claude con la descrizione dell'impatto e l'immagine come vision block. Claude genera l'HTML della versione modificata.
5. Puppeteer renderizza l'HTML in PNG (1280×800) e il PNG in base64 viene salvato in `impact_prototypes`.
6. Il frontend riceve il base64 e mostra la comparazione **AS-IS / Modified** side by side.
7. Il pulsante **Download PNG** scarica l'immagine. **Re-generate** permette di ricaricare un nuovo AS-IS.

7.2 Comportamento accordion

Il componente `ImpactPrototype` rimane sempre montato nel DOM (anche quando l'accordion è chiuso), usando `class="hidden"` invece di unmounting condizionale. Questo garantisce che la generazione in corso non venga persa se l'utente chiude e riapre l'accordion.

7.3 Persistenza

Il prototipo generato viene salvato nel DB. Alla riapertura del tab, il componente fa automaticamente una chiamata GET per recuperare l'eventuale prototipo già generato in precedenza. Una nuova generazione sovrascrive (upsert) il precedente.

TIMEOUT

Generazione: 4 minuti (Claude + Puppeteer). L'utente vede uno spinner animato. In caso di errore, il messaggio viene mostrato estratto correttamente dalla risposta Axios (`err.response.data.error`).

FILE TEMPORANEI

L'immagine AS-IS caricata viene salvata in `./tmp-uploads/` e cancellata automaticamente dopo l'uso, in un blocco `finally`. Non viene archiviata permanentemente.

SEZIONE 8

Funzionalità — Risk Assessment

Il Risk Assessment è pensato per il contesto in cui si sta estendendo un'applicazione a una nuova country/cliente, e si vogliono prevenire i problemi noti di un deployment precedente. Esempio: KFC Austria ha generato 120 defect in UAT; si sta preparando il deployment KFC Slovakia e si vuole capire quali aree monitorare di più.

8.1 Input richiesti

- **File defect** — export da ALM (Excel .xlsx/.xls o CSV). Il file deve contenere almeno una colonna per la priorità/severità e una per la descrizione del defect.
- **Contesto sorgente** — nome del progetto/country da cui provengono i defect (es. "KFC Austria")
- **Contesto target** — nome del deployment per cui si vuole la previsione (es. "KFC Slovakia")

8.2 Flusso multi-step asincrono

A differenza dell'analisi funzionale, il Risk Assessment esegue due chiamate a Claude in sequenza, con feedback progressivo all'utente:

1. **Parsing file** — il file Excel/CSV viene letto e convertito in righe JSON. Limite: 200 righe inviate a Claude (cap 30.000 chars).
2. **Categorizzazione** — prima chiamata Claude: raggruppa i defect per area funzionale, analizza la distribuzione per priorità, identifica i top issue ricorrenti e i pattern/root cause.
3. **Raccomandazioni** — seconda chiamata Claude: sulla base della categorizzazione, genera le aree di rischio per il deployment target con livello (high/medium/low), razionale e azione preventiva specifica.
4. Il risultato unificato viene salvato in `risk_assessments.result_json` e visualizzato nel frontend.

8.3 Output visualizzato

- **Overall Risk Badge** — livello di rischio complessivo (high/medium/low) con icona
- **Summary** — 3-4 frasi di sintesi del profilo di rischio per il target
- **Defect Categories** — bar chart orizzontale con count e percentuale per area funzionale
- **Priority Distribution** — bar chart con distribuzione per priorità/severità
- **Top Recurring Issues** — tabella con i 5-8 problemi più impattanti
- **Observed Patterns** — 4-6 pattern/root cause identificati
- **Risk Areas & Recommendations** — cards per area: rationale + azione preventiva concreta

8.4 Navigazione

Il Risk Assessment è accessibile dalla pagina di dettaglio progetto tramite il pulsante **Risk Assessment** nell'header. È una pagina separata (`/projects/:id/risk-assessment`) con pannello storico a sinistra e risultati a destra, analogo alla struttura del progetto principale.

SEZIONE 9

API REST — Riferimento

9.1 Progetti — `/api/projects`

GET `/api/projects`
Lista tutti i progetti ordinati per data decrescente

GET `/api/projects/:id`
Dettaglio progetto con files[] e analyses[] embedded

POST `/api/projects`
Crea nuovo progetto — body: { name, description }

PATCH `/api/projects/:id`
Aggiorna nome/descrizione — body: { name?, description?, status? }

DELETE `/api/projects/:id`
Elimina progetto + file su disco + cascade DB

9.2 File — `/api/files`

GET `/api/files/:projectId`
Lista file del progetto

POST `/api/files/:projectId/upload`
Upload file — multipart/form-data: { file, bucket: as-is|to-be|business-rules }

DELETE `/api/files/:projectId/:fileId`
Elimina file dal DB e dal filesystem

GET `/api/files/:projectId/:fileId/preview`
Serve il file per anteprima (inline, con Content-Type corretto)

9.3 Analisi — `/api/analysis`

GET `/api/analysis/:projectId`
Lista analisi del progetto

POST `/api/analysis/:projectId/run`
Avvia analisi — risponde 202; esecuzione asincrona in background

GET `/api/analysis/:projectId/:analysisId`
Dettaglio singola analisi

DELETE `/api/analysis/:projectId/:analysisId`
Elimina analisi (cascade su impact_prototypes)

POST `/api/analysis/:projectId/:analysisId/impact-prototype`

Genera prototipo per impatto — multipart: { file (immagine), impactId, impactArea, impactDescription }

GET `/api/analysis/:projectId/:analysisId/impact-prototype/:impactId`

Recupera prototipo salvato — risponde 404 se non ancora generato

9.4 Risk Assessment — `/api/risk`

GET `/api/risk/:projectId`

Lista risk assessment del progetto

POST `/api/risk/:projectId/run`

Avvia assessment — multipart: { file (.xlsx/.xls/.csv), sourceContext?, targetContext? } — risponde 202

GET `/api/risk/:projectId/:assessmentId`

Dettaglio singolo assessment (con progress_step se in running)

DELETE `/api/risk/:projectId/:assessmentId`

Elimina assessment

SEZIONE 10

Sicurezza e limitazioni MVP

10.1 Sicurezza implementata

- **Nessuna chiave API nel codice:** `ANTHROPIC_API_KEY` solo in `.env`, gitignored.
- **Sanitizzazione HTML:** il prototipo HTML generato da Claude viene sanitizzato con DOMPurify prima del rendering in iframe (codebase precedente — ora si usa direttamente il PNG, eliminando il rischio XSS).
- **Validazione MIME type:** multer filtra i tipi di file accettati sia per l'upload documenti sia per il prototipo e il risk assessment.
- **Prepared statements:** tutte le query SQLite usano prepared statements — nessuna interpolazione di stringhe SQL.
- **Cancellazione file temporanei:** i file caricati per generazione prototipo e risk assessment vengono cancellati in `finally` dopo l'uso.
- **CORS:** configurato solo per `http://localhost:5173`.

10.2 Limitazioni MVP

- **Nessuna autenticazione:** l'applicazione è single-user, nessun login. Chiunque abbia accesso alla porta 3001 può leggere tutti i progetti.
- **SQLite locale:** non supporta multi-utente concorrente in scrittura; in produzione multi-tenant servirebbe PostgreSQL.
- **Nessuna streaming:** Claude risponde tutto in una volta; analisi lunghe richiedono attesa senza feedback token-by-token.
- **File su filesystem locale:** in caso di restart del server o reset del filesystem, i file caricati sono persi (non c'è object storage).
- **Risk Assessment cap:** vengono inviati massimo 200 defect a Claude. File con migliaia di righe vengono troncati.
- **Nessun export:** non è possibile esportare l'analisi in PDF/Word dall'interfaccia (in roadmap).

SEZIONE 11

Roadmap evolutiva

Priorità	Funzionalità	Razionale
Alta	Streaming risposte Claude	Mostrare i token man mano che arrivano, eliminando la percezione di attesa.
Alta	Export analisi in PDF/Word	Il functional analyst deve consegnare un documento al cliente.
Alta	Auth layer (JWT o Clerk)	In ambienti condivisi, necessario per isolare i progetti per utente.
Media	Vista comparativa multi-progetto	Confrontare analisi di versioni diverse dello stesso prodotto.
Media	Integrazione Jira / Azure DevOps	Creare ticket/work item direttamente dagli impatti identificati.
Media	RAG su documenti grandi	Per documenti superiori ai limiti del contesto, usare embedding + retrieval.
Bassa	PostgreSQL migration	Per deployment multi-utente in produzione su infrastruttura condivisa.
Bassa	Risk Assessment aumentato	Gestire più di 200 defect tramite chunking o RAG; aggiungere grafici interattivi.

Note finali: Il Functional Accelerator è un MVP funzionante e deployabile in ambienti di consulting interni. L'architettura è intenzionalmente semplice (SQLite, single process, no auth) per consentire setup in pochi minuti su qualsiasi laptop. Le evoluzioni in roadmap seguono la priorità dettata dall'uso reale in progetti di delivery.